

演示视频脚本 — 砚台（网络小说智能生成平台）

总时长：约 8-10 分钟

结构：Part 1 Agent 应用（产品展示，4-5min）+ Part 2 实现架构（技术讲解，3-4min）

录制建议：1080p，前端界面全屏录制，关键架构部分可用 Keynote/PPT 配合录屏

Part 1 — Agent 应用演示（产品展示风格）

目标：让观众感受到这是一个"能用、好用、有温度"的 AI 创作助手，而非冷冰冰的预设工作流。

情绪基调：创作者视角，解决痛点，展现流畅体验。

【镜头 1】开场 — 直接入题（20s）

项目	内容
画面	直接录屏展示浏览器，地址栏输入 localhost:5173，进入砚台 Dashboard 首页。书架视图下已有 2-3 个预置项目卡片（如《雾都猎魔人》《星辰大海》），画面整洁。
旁白	"大家好，这是砚台——一个基于多智能体协作的网络小说 AI 辅助创作平台。它不只是一个聊天机器人，而是一个有世界观专家、角色专家、情节专家和审稿人组成的 AI 创作团队。接下来我用一个完整的小说创作流程，演示它的核心能力。"
转场	鼠标点击"新建项目"按钮，进入项目创建弹窗。
时长	20s

【镜头 2】场景一：从零构建世界观（1min 30s）

项目	内容
画面	① 展示 Dashboard 书架视图，点击"新建项目"，输入项目名称《雾都猎魔人》，类型：都市奇幻。 ② 左侧导航进入"世界观"页面，当前为空。 ③ 点击右下角悬浮 AI 对话按钮（蓝色机器人图标），弹出聊天抽屉。 ④ 用户输入："帮我设计一个现代都市背景下的猎魔人世界观，要有独特的力量体系和地理设定"。
旁白	"这是砚台，一个 AI 辅助创作平台。我先创建一个都市奇幻项目。现在，我什么都不用写，直接告诉 AI 我的想法。"
画面 (续)	⑤ 聊天界面开始出现时间线：第一条——"delegate_to_world_builder — 正在执行..."；紧接着出现 Thinking 内容："用户需要现代都市猎魔人世界观，我需要先查询现有内容..."；然后出现工具调用链：query_content → get_outline_tree → create_world_setting。 ⑥ 时间线节点逐个变绿（完成状态），最终 AI 回复一段完整的设定描述，包含"雾都结社""以太污染""猎魔人纹章"等原创概念。 ⑦ 世界观页面自动刷新，出现两张卡片：《雾都结社档案》《以太污染与力量体系》。

项目	内容
旁白 (续)	"注意看右侧的时间线——AI 没有直接瞎编。它先查询了项目里现有的内容，确认没有冲突，然后调用世界观构建 Agent，生成了两份结构化设定文档。所有内容自动保存，我随时可以点开编辑。"
时长	1min 30s

【镜头 3】场景二：角色塑造与一致性保障（1min 30s）

项目	内容
画面	① 用户继续输入："基于刚才的世界观，给我设计三位主角：一位叛逆的年轻猎魔人、一位神秘的女巫、一位亦正亦邪的结社高层"。② 时间线显示：delegate_to_character → query_content(domain="world") → get_content(某世界观ID) → create_character（三次）。③ 角色页面自动刷新，出现三张角色卡片，头像占位但信息完整。④ 点击第一张卡片"陆昭"，进入 Vditor 分屏编辑器，左侧 Markdown 源码，右侧渲染后的角色档案，包含外貌、性格、背景故事、与以太污染的关联。⑤ 用户滚动查看档案，高亮显示"以太敏感体质"与世界观设定中的"以太污染"形成呼应。
旁白	"接下来，我让 AI 生成三位主角。同样，它先去读了世界观设定，确保角色能力与力量体系自洽。你看，主角陆昭的'以太敏感体质'，直接呼应了世界观的'以太污染'设定——这不是巧合，是 Agent 主动查资料后做的设计决策。"
画面 (续)	⑥ 用户返回聊天，追问："陆昭的性格太温和了，让他更叛逆一些，再加一段和父亲的冲突"。⑦ AI 回复修改后的档案，用户点击"角色"页面，陆昭卡片已更新。
旁白 (续)	"修改也只需要一句话。AI 不是模板填空，它理解上下文，能根据我的反馈自我调整。"
时长	1min 30s

【镜头 4】场景三：大纲生成、正文撰写与智能审阅（1min 30s）

项目	内容
画面	① 用户输入："基于世界观和角色，生成一部 3 卷结构的小说大纲，第一卷聚焦陆昭觉醒"。② 时间线显示 delegate_to_plot → 多次 query_content/get_content → create_outline（root→volume→chapter 层级创建）。③ 切换到大纲编辑器页面，展示树形结构：根节点《雾都猎魔人》→ Volume 1《觉醒之雾》→ 5 个 chapter 节点。④ 用户拖拽调整第 2、3 章顺序，演示大纲可人工干预。⑤ 用户选中 Chapter 1《雨夜初醒》，输入："把这一章写成正文，3000 字左右，要有动作场面和情绪张力"。⑥ 时间线显示 delegate_to_chapter → get_outline_tree → get_content(世界观+角色) → write_chapter。⑦ 章节页面出现新章节卡片，字数统计显示 3127 字。点击进入编辑器，正文以 Markdown 格式呈现，有场景描写、对话、内心独白。
旁白	"大纲生成是 Agent 的拿手好戏。它会严格遵循 root→volume→chapter 的层级结构，而且我可以随时拖拽调整——AI 辅助，但创作主权在我。写完大纲，我挑一章让 AI 扩写成正文。它自动拉取了世界观、角色档案和大纲细节，写出来的内容不是泛泛而谈，有具体的场景、对话和情绪节奏。"

项目	内容
画面 (续)	⑧ 用户在聊天中输入: "审阅一下第一章"。⑨ 时间线显示 delegate_to_review → get_content(chapter) → query_content(world+character) → review_content。⑩ AI 返回审阅报告: 综合评分 8.2/10, Issues 列表指出"第 3 段以太浓度的描述与世界观设定中'普通人不可感知'存在轻微矛盾", Suggestions 给出具体修改建议。⑪ 用户打开章节编辑器, 根据建议修改了一处描述。
旁白 (续)	"最后, 我请审阅 Agent 把关。它不仅打分, 还指出了世界观一致性的小问题——这正是人类编辑最难兼顾的。AI 在这里不是写手, 而是 24 小时在线的责编。"
时长	1min 30s

【镜头 5】结尾 — 创作的本质 (30s)

项目	内容
画面	快速回放: 项目创建 → 世界观卡片 → 角色卡片 → 大纲树 → 章节正文 → 审阅报告。最后定格在聊天界面, 时间线全部收起, 只剩 AI 的最后一句话: "需要我继续扩展第二卷的剧情吗?"
旁白	"AI 不会取代创作, 但它能替我们记住那些容易遗忘的细节, 让我们把精力留给真正重要的东西——讲故事。"
画面 (终)	出现项目 Logo 和 Slogan: 砚台 — 让想象落地。
时长	30s

Part 2 — 实现架构介绍 (技术讲解风格)

目标:	向技术评审展示系统架构的深度、Agent 设计的合理性、关键技术的创新点。
情绪基调:	专业、清晰、自信, 突出设计决策而非简单堆叠技术名词。

【镜头 6】总体架构概览 (1min)

项目	内容
画面	PPT 展示总体架构图 (6 层): Client Layer (React + Ant Design X) → Gateway (Nginx, 可选) → Application Layer (FastAPI) → Services Layer (ContentService Facade + 领域服务) → AI Engine Layer (Orchestrator + Sub-Agents + Tool Factory) → Persistence Layer (SQLite + Markdown Files + Checkpoint)。每层用不同颜色区分。底部标注: "单机部署 · 无需云服务器"。

项目	内容
旁白	"这是系统的总体架构。前端基于 React 18 和 Ant Design X 构建，后端是 FastAPI 服务，全部运行在本地。这是一个单机部署项目——创作者在自己的电脑上安装，数据完全本地存储，无需云服务器，也无需联网（除调用 LLM API 外）。后端的 AI 编排引擎是核心，基于 LangGraph 的多智能体协作系统。持久层采用混合存储：SQLite 存元数据，Markdown 文件存创作内容本体，LangGraph Checkpoint 存会话状态，全部落在本地磁盘。"
时长	1min

【镜头 7】多 Agent 协作架构（1min 30s）

项目	内容
画面	PPT 切换为 Agent 架构图。中心是 Orchestrator (Supervisor)，外围 5 个子 Agent：world_builder、character、plot、chapter、review。箭头标注 Handoff 关系。下方标注："Subagents-as-Tools Pattern"。
旁白	"AI 引擎层采用 Subagents-as-Tools 模式。Orchestrator 是一个 ReAct Agent，它通过系统提示词中定义的 5 个 handoff 工具——delegate_to_world_builder、delegate_to_character 等——来决定把任务路由给哪个子 Agent。每个子 Agent 本身也是一个独立编译的 LangGraph 图，拥有专属的系统提示词和工具集。"
画面 (续)	动画演示一次完整调用：User Input → Orchestrator 思考"这是世界观相关请求" → 调用 delegate_to_world_builder → 子 Agent 执行 query_content → get_content → create_world_setting → 返回结果 → Orchestrator 汇总回复用户。
旁白 (续)	"为什么不用单一 Agent？因为小说创作涉及高度专业化的领域知识。让世界观专家去写打斗场面，或者让写手去设计魔法体系，都不合理。多 Agent 架构让每个专家做自己擅长的事，Orchestrator 负责统筹协调——这正是人类出版社的分工模式。"
时长	1min 30s

【镜头 8】三大关键技术决策（1min 30s）

项目	内容
画面	PPT 分三栏，每栏一个技术点，配简图。
旁白	"这里有三个支撑整个系统的关键技术决策，我想重点展开。"

技术点 1：闭包绑定 Project ID（20s）

| 画面 | 左侧栏放大，显示代码示意图：`def create_world_tools(project_id): @tool def create_world_setting(name, content): ... asyncio.run(_create())`。红色高亮 `project_id` 在闭包中被捕获。 |

| 旁白 | "第一，项目隔离。所有工具函数通过工厂模式创建，project_id 通过 Python 闭包绑定到每个工具内部。这意味着：没有模块级单例，不存在线程安全问题，更不可能出现项目 A 的工具误操作项目 B 的数据。" |

技术点 2: SSE v2 Streaming + 子图事件透传 (40s)

| 画面 | 中间栏放大，显示 SSE 事件流图。backend graph.astream(..., stream_mode=["messages", "updates", "custom"], subgraphs=True, version="v2") → 前端 EventParser 分类处理 → 时间线渲染。标注不同事件类型的颜色：message (蓝)、tool_start/end (橙)、thinking (紫)、error (红)。 |

| 旁白 | "第二，状态可视化。后端使用 LangGraph v2 Streaming API，开启 subgraphs=True，这样子 Agent 内部的事件也能透传到前端。前端的时间线不是伪造的进度条，而是真实的事件映射：on_chat_model_stream 变成 token 流，on_tool_start/end 变成工具节点，Claude 的 thinking 内容也会实时展示。用户看到的'正在查询世界观...'，就是 Agent 此刻正在做的事。" |

技术点 3: SkillMiddleware 动态技能注入 (30s)

| 画面 | 右侧栏放大，显示 Skill 模板格式 (YAML Frontmatter + Markdown 正文)，以及 SkillMiddleware 在模型调用前自动拼接技能描述的流程图。 |

| 旁白 | "第三，技能系统。创作者可以把自己的写作方法论沉淀为 Skill 模板——比如'黄金三章写作法'——以 Markdown + YAML 格式保存。SkillMiddleware 在每个 Agent 调用 LLM 前，自动将匹配 domain 的技能描述注入系统提示词。更关键的是，Orchestrator 还具备运行时创建新技能的能力，Agent 在创作过程中发现有效的方法论，可以自动沉淀下来供未来复用。" |

| 时长 | 1min 30s |

【镜头 9】错误处理、持久化与扩展性 (1min)

项目	内容
画面	PPT 快速展示三张图：① AIError 分类映射表 (RateLimit/Timeout/Network→重试；Auth/BadRequest→立即报错)；② 指数退避公式 <code>delay = base_delay * 2^(attempt-1) * jitter</code> ；③ Checkpoint 恢复流程 (会话中断 → 重启 → aget_state → 恢复上下文)。
旁白	"可靠性方面，我们实现了完整的错误分类和恢复机制。LLM 异常被映射为结构化的 AIError，可重试错误自动进行指数退避重试，不可重试错误通过 SSE error 事件实时通知前端。会话状态通过 LangGraph Checkpoint 持久化到 SQLite，即使服务器重启，用户回到聊天也能无缝续聊。"
画面 (续)	出现 Roadmap：Human-in-the-Loop (关键写操作人工确认) → LangSmith 全链路追踪 → 多 LLM 供应商支持 (OpenAI/本地模型) → 可选 SaaS 化部署。
旁白 (续)	"目前的架构已经为未来扩展预留了充足空间：HITL 人工审批节点、LangSmith 追踪、多模型供应商切换。当然，如果未来要从单机工具扩展为云服务，只需要把 SQLite 替换为 PostgreSQL，加上用户认证层——核心 Orchestrator 逻辑完全不用动。"
时长	1min

【镜头 10】结尾 — 技术总结（15s）

项目	内容
画面	PPT 显示一句话总结："多智能体分工 + 实时状态可视化 + 项目级安全隔离 = 可信赖的 AI 创作引擎"。下方附 GitHub/GitLab 仓库二维码。
旁白	"总结一下：多智能体分工让专业的人做专业的事，实时状态可视化让用户信任 AI 的每一步决策，项目级安全隔离确保创作者的数据完全本地、安全可控。这是一个运行在创作者自己电脑上的 AI 创作引擎——无需云服务器，无需担心隐私泄露。这就是砚台的技术内核。"
时长	15s

附录：录制技术建议

项目	建议
屏幕录制	使用 OBS Studio 或 ScreenFlow，前端界面部分建议 1920×1080，PPT 部分同样 1080p
前端演示	提前准备好 mock 数据，确保 AI 响应流畅。如果真实调用 LLM 太慢，可预先录好关键流程
时间线高亮	录制时鼠标跟随工具时间线的展开/完成过程，帮助观众理解状态流转
PPT 动画	Agent 架构图建议用 Keynote 的 Magic Move 或 PPT 的平滑切换，展示 Handoff 流程
旁白语速	产品展示部分稍慢，留给观众看界面；技术讲解部分可稍快，信息密度高
BGM	产品展示部分可配轻音乐（音量低于旁白 20dB），技术讲解部分建议无音乐